

Corso di Laurea in Ingegneria e Scienze Informatiche

Progettazione di un configuratore di vendite

Tesi di laurea in:
BASI DI DATI

Relatore

Prof.sa Annalisa Franco

Candidato

Cavina Mattia

Sommario

To be written.

Indice

Sommario	iii
1 Introduzione	1
2 Situazione As-Is	3
2.1 Premessa	3
2.2 Situazione infrastrutturale aziendale	3
2.3 Attuali software utilizzati	6
2.4 Organizzazione aziendale e flussi di lavoro	6
3 Raccolta dei requisiti	9
3.1 Scelta dei candidati	9
3.2 Modalità di svolgimento interviste	10
3.3 Risultati interviste condotte	11
3.3.1 Rispetto all'attuale sistema	11
3.3.2 Richeste avanzate	12
4 Analisi dei requisiti	15
4.1 Requisiti funzionali	15
4.2 Requisiti non funzionali	17
5 Progettazione dell'architettura	19
5.1 Premessa sulle scelte architetturali	19
5.2 Deployment	20
5.3 Package	21
5.3.1 Interfaccia al database	22
5.3.2 Libreria di componenti view	23
5.3.3 Modulo main e view	23
5.4 Flussi di lavoro	25

6	Scelte progettuali	27
6.1	Scelta modello di sviluppo	27
6.2	Suddivisione e contenuto dei rilasci	27
6.3	Scelta framework ORM	29
6.4	Scelta del framework grafico	30
6.5	Scelta del pattern di progettazione	30
7	Progettazione del database	33
7.1	Esportazione vecchio database	33
7.2	Denominazione tabelle e campi	33
7.3	Utilizzo di SSIS nel processo di migrazione	34
7.4	Ristrutturazione database	34
7.4.1	Eliminazione ridondanze e normalizzazione	34
7.4.2	Creazione tabelle di lookup	35
7.4.3	Aggiunta tabelle non presenti	35
7.4.4	Importazione dei dati e Stored Procedure	39
7.4.5	Schema logico	39
8	Creazione Model	41
8.1	Mapping delle classi	41
8.2	Struttura del package	41
8.2.1	Classi di Test	42
8.2.2	Repository	43
8.3	Gestione dei dati e delle transazioni	44
9	Creazione View	45
9.1	Utilizzo di Figma per il disegno delle interfacce	45
9.2	Creazione delle Factory per le View	46
9.3	Scelta delle classi di ViewModel	47
10	Modulo configurazione prodotto	49
10.1	Scopo del modulo	49
10.2	Algoritmo di configurazione	49
10.2.1	Scelta macrogruppo da configurare	50
10.2.2	Implementazione delle regole	51
10.3	Interfaccia grafica lato utente ed amministratore	52
10.3.1	Lato utente	52
10.3.2	Lato amministratore	54

	57
Bibliografia	57

Elenco delle figure

2.1	Schema infrastruttura aziendale.	5
2.2	Schema degli stati di una offerta commerciale	8
4.1	Requisiti funzionali raccolti.	16
4.2	Requisiti non funzionali raccolti.	17
5.1	Diagramma dei deployment	21
5.2	Diagramma dei package	24
5.3	Nuovo diagramma degli stati di un contratto	25
6.1	Diagramma funzionamento MVVM	31
7.1	Schema Entity Relationship (ER) dei prodotti	36
7.2	Schema ER delle lingue	37
7.3	Schema logico del database	40
9.1	Prototipo Home Figma	46
9.2	Interfaccia Desktop e Mobile	46
10.1	Albero di scelta prodotto	51
10.2	Modello ER configuratore	52
10.3	Configurazione lato utente	53
10.4	Risultato configurazione lato utente	53
10.5	Aggiunta regole	54
10.6	Test regole lato amministratore	55

Capitolo 1

Introduzione

Il progetto di tesi è volto alla progettazione ex-novo del programma di vendita dell'azienda Cepi S.p.a., azienda con una produzione di tipo Engineering To Order (ETO) specializzato nella progettazione di impianti di stoccaggio custom. La tipologia di produzione ETO ha portato ad un profondo studio della situazione odierna, delle sue esigenze attuali e future tenendo aperte le possibilità di ampliamenti. L'analisi ha comportato un sostanziale lavoro di programmazione e raccolta dati per coprire nel modo più ampio i bisogni aziendali. Questa prima fase è stata condotta inizialmente attraverso la selezione di figure chiave scelte per mansione o/e ruolo, che potessero dare una visione dei bisogni del proprio o altrui reparto e successivamente sottoponendoli ad interviste. Sono state condotte 14 interviste, di cui 8 in modalità vis a vis, mentre le restanti 6 attraverso la dimostrazione dell'attuale applicativo utilizzato. Il risultato delle interviste condotte, ha fatto emergere attuali e nuove necessità architetture da affrontare come il cross-platform, l'utilizzo offline del database e la necessità di costruire offerte in modo più agile e strutturato. Questa prima fase ha inoltre sottolineato come il progetto non debba solo sostituire l'attuale sistema, ma debba inserirsi in un contesto di rinnovamento più ampio, coordinandosi con altri programmi aziendali e relativi database in ottica "Single Responsibility Principle (SRP)" eleggendo dei master data che siano i soli responsabili della creazione di certi dati. Il SRP ha plasmato notevolmente la costruzione del database ricreando, dove possibile, le strutture pre-esistenti da cui si necessitava recuperare i dati mediante reverse en-

gineering. 7.4.4 L'intero sistema è stato pensato per appoggiarsi all'infrastruttura esistente aziendale, come SQLServer[Micd], ma studiando delle logiche di flessibilità e ampliamento dinamico studiando accuratamente le relazioni tra le tabelle. Si è reso necessario cercare e scegliere inoltre un framework per la parte grafica che potesse soddisfare il deployment cross-platform. Questo requisito è emerso in modo chiaro dalle interviste, in quanto ci sono situazioni in cui il personale aziendale si trova a dover lavorare in mobilità, come le fiere di settore. La costruzione delle interfacce grafiche, è stata supportata dal sito "Figma"[Fig] che ha permesso una progettazione in ottica mobile first.⁹ La scelta accurata dei framework e le strutture dati nella fase iniziale, ha pesato molto in termini di tempo e progettazione, per garantire al richiedente un sistema che non rischiasse né il debito tecnico, né la preclusione a sviluppi futuri. La pianificazione dei primi test con gli utenti, dei rilasci delle demo e del metodo di progettazione sono state cruciali per dettare tempi e priorità degli item dovendo coprire in modo esaustivo almeno il core del progetto.

Capitolo 2

Situazione As-Is

2.1 Premessa

La situazione infrastrutturale aziendale è stata studiata dal sottoscritto nel corso del tirocinio formativo. La raccolta dei dati è stata svolta mediante interviste e studio dell'attuale sistema informatico. Nelle sezioni che seguono introdurrò brevemente l'As-Is.

2.2 Situazione infrastrutturale aziendale

L'infrastruttura informatica aziendale si appoggia su server fisici allocati internamente e gestiti mediante Active Directory. I server fisici sono, a loro volta virtualizzati per gestire separatamente le memorie in base alla finalità garantendo isolamento delle informazioni e permessi di accesso. Ogni reparto ha accesso a determinati server virtuali in cui gestisce in autonomia i propri contenuti. I database dei principali programmi sono gestiti all'interno di un server dedicato dove è installato un motore SQLServer.[Micd]. I principali database aziendali sono:

- **AdHoc[Zuc]**: l'Enterprise Resource Planning (ERP) aziendale, programma che gestisce le attività aziendali come gli approvvigionamenti e la movimentazione dei prodotti, le vendite, le fatturazioni nonché i cicli produttivi.

- **Fusion[[Rul](#)]**: il Product Lifecycle Management (PLM) aziendale, programma volto alla gestione dei prodotti meccanici, dalla loro creazione all'archiviazione dei disegni con relative distinte dei componenti. Altre funzioni applicate sui prodotti sono la gestione delle revisioni o versionamenti, ed infine la loro fine produzione.
- **BombConfine**: database ponte tra i database esterni e quello dell'ERP. All'interno sono presenti tabelle di frontiera scritte da programmi esterni e che il gestionale usa per importare i dati.

La sicurezza aziendale è sostenuta da Network Address Translation (NAT) e Firewall che impediscono l'accesso dall'esterno se non tramite Virtual Private Network (VPN). L'infrastruttura aziendale è visibile in figura [2.1](#).

2.2. SITUAZIONE INFRASTRUTTURALE AZIENDALE

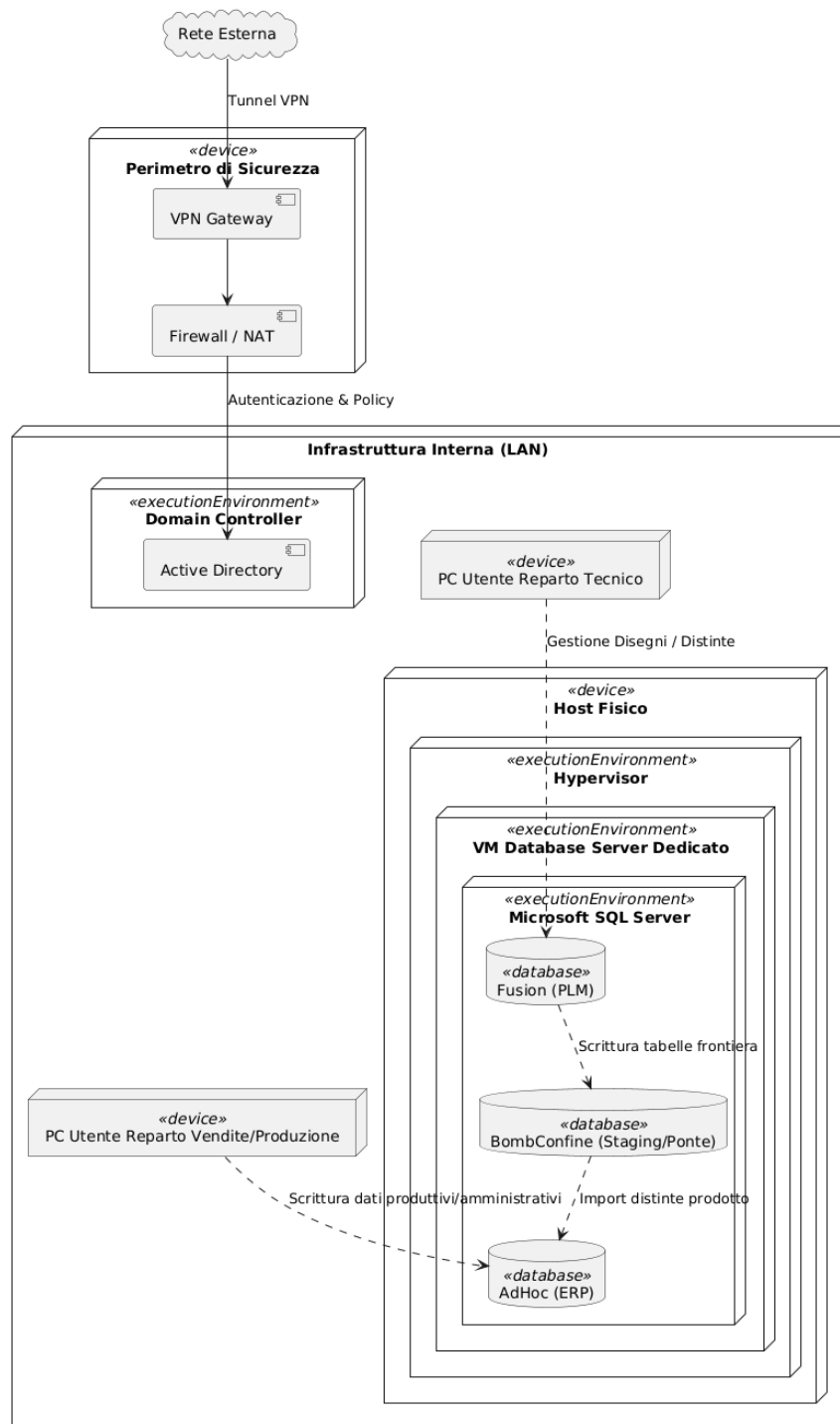


Figura 2.1: Schema infrastruttura aziendale.

2.3 Attuali software utilizzati

Nel capitolo precedente abbiamo fatto cenno ai database ed ai relativi programmi che gestiscono quanto concerne l'ERP e il PLM. Per quanto riguarda le vendite, punto focale dell'intervento richiesto, l'azienda utilizza attualmente un'applicazione sviluppata internamente. L'attuale sistema sviluppato in Access, programma della suite Microsoft Office, comprende al suo interno sia il frontend che il backend. Il software è stato implementato circa 30 anni fa ed ha subito numerosi aggiornamenti e modifiche nel tempo per adattarsi alle esigenze commerciali e all'evoluzione del mercato. L'applicazione consente di creare, modificare e gestire i contratti con i clienti, includendo funzionalità quali la gestione degli sconti, la personalizzazione delle offerte e la stampa dei documenti. Tuttavia, nonostante le funzionalità implementate, il software presenta significative criticità che ne limitano l'efficienza e l'usabilità, come confermato dalle interviste condotte con gli utenti. Tali criticità includono instabilità operativa, mancanza di standardizzazione, codifiche obsolete, difficoltà negli aggiornamenti di prezzo e assenza di integrazione con altri sistemi aziendali. Di conseguenza, è indispensabile sviluppare un nuovo programma che affronti tali problematiche e migliori l'esperienza degli utenti, preservando nel contempo le funzionalità essenziali del sistema attuale. Il programma Access è stato fondamentale in questi anni per l'utilizzo in zone offline. Questa funzionalità è stata favorita dalla natura del programma stesso, il quale racchiude in un solo file database e frontend, copiandolo in locale e sincronizzandolo una volta tornato in sede.

2.4 Organizzazione aziendale e flussi di lavoro

Il flusso lavorativo è risultato non essere un aspetto secondario nello sviluppo richiesto. Esso si deve interporre tra il reparto commerciale, ovvero addetti alle vendite, i reparti progettuali e produttivi come uffici tecnici, logistica ed amministrazione. L'attuale flusso di lavoro analizzato pone il reparto commerciale come punto di partenza dell'intero flusso che arriva fino al post-vendita. Il contratto, creato da tale reparto, dopo essere stato stipulato viene inviato al reparto amministrativo che ne importa i dati all'interno del gestionale. Simultaneamente esso

viene mandato al reparto project manager che lo revisiona tecnicamente in caso di errori progettuali. Dopo la revisione tecnica, il contratto viene spedito al reparto logistico che preventivamente verifica i dati di destinazione ed organizza le spedizioni sulla base del venduto. Il flusso di lavoro può essere visto in figura 2.2. A causa del sistema obsoleto utilizzato ed il debito tecnico accumulato negli anni, l'attuale sistema è isolato rispetto ai programmi utilizzati dall'azienda. Quest'ultimo punto fa parte delle richieste imprescindibili da risolvere. L'integrazione nell'ecosistema informatico passa prima di tutto dall'unificazione dei codici, ovvero quelli a listino con i prodotti a gestionale, ma anche creando un flusso di notifiche tra i vari ambiti.

Capitolo 3

Raccolta dei requisiti

3.1 Scelta dei candidati

La raccolta requisiti si è svolta mediante la selezione di figure aziendali con distinti ruoli ed esperienza. Queste figure selezionate sono state poi sottoposte ad una intervista sia in modalità vis a vis, che in alcuni casi da remoto per favorire la registrazione del meeting. I candidati possono essere raggruppati in soci aziendali, utilizzatori, e non utilizzatori. All'interno degli ultimi due gruppi sono state selezionate, per far fronte ai basias dovuti all'abitudine, candidati sia con esperienza pluriennale che neoassunti. In totale sono state intervistate 14 persone così suddivise:

Utilizzatori abituali:

- **Agenti commerciali esterni:** 2 persone
 - 1 agente con esperienza limitata.
 - 1 agente con esperienza consolidata.
- **Commerciali interni:** 4 persone
 - 1 specializzato in nuovi impianti.
 - 1 specializzato in ricambi.

- 2 con competenze trasversali, sia per quanto concerne nuovi impianti che post-vendita.

Utilizzatori saltuari:

- **Soci aziendali con responsabilità:** 3 persone
 - 1 responsabile delle vendite.
 - 1 responsabile tecnico.
 - 1 responsabile amministrativo.
- **Direttore di produzione:** 1 persona

Non utilizzatori:

- **Impiegati tecnici:** 4 persone
 - 1 Project manager.
 - 1 Responsabile logistica.
 - 1 Progettista meccanico.
 - 1 Progettista elettrico.

L'ordine delle interviste svolte non è stato lineare, ma si è optato di partire dalla proprietà, per capire la direzione ed aspettative del progetto. Le restanti interviste hanno seguito un ordine logico, ma al contempo sparso, alternando utilizzatori e non utilizzatori, con l'obiettivo di non saturare ogni singolo reparto. Questa modalità, ha consentito di fare da guida per le interviste successive.

3.2 Modalità di svolgimento interviste

Le interviste, come esplicitato in precedenza, si sono svolte singolarmente sia in presenza che da remoto. Con il permesso degli interlocutori sono state registrate, anche solo l'audio, per poi essere rielaborate e trascritte con supporto di Intelligenza Artificiale (IA) generativa. La scelta di utilizzare la modalità fisica o da remoto

è stata dettata dalla necessità. Per gli utilizzatori abituali si è reso necessario registrare visivamente come utilizzano l'attuale applicativo. Tramite questa modalità si è potuto raccogliere i flussi di utilizzo e le principali funzioni presenti da mantenere. Nei restanti incontri si è preferito un approccio in presenza con l'interlocutore, facilitato la raccolta di requisiti e consigli sul progetto.

Le prime interviste sono state supportate da una scaletta di domande, preparate sulla base dell' AS-IS 2 che hanno guidato l'incontro. Con il progredire delle interviste, è stato adottato un approccio più flessibile che ha affinato i risultati delle interviste in previsione delle successive. Questa modalità ha consentito di ricavare i requisiti fondamentali ordinati per importanza.

3.3 Risultati interviste condotte

3.3.1 Rispetto all'attuale sistema

Funzionalità da mantenere

Dalle interviste condotte, è emersa l'importanza di mantenere una serie di funzionalità, quali:

- Suddivisione del contratto in capitoli che raggruppano porzioni di esso.
- Gestione delle scontistiche a cascata, ovvero la possibilità di inserirle rispetto alla singola voce contrattuale, nel capitolo, o dell'offerta stessa.
- Gestione di voci o capitoli opzionali, che non concorrono al prezzo offerto, ma ad uno opzionale.
- I campi del database che attualmente concorrono alla creazione dei report, sono da mantenere.
- Creazione reportistica formato PDF del contratto digitale. Da gestire, come ora, il multilingua del report stesso.
- Esportazione all'ERP delle voci contrattuali presenti nell'ordine.

Criticità emerse

Mediante le dimostrazioni di utilizzo si è potuto raccogliere le criticità da attenzionare, come:

- Blocchi frequenti in condizione di concorrenza. Se più utenti utilizzano in contemporanea il file si generano deadlock.
- Non è presente una gestione centralizzata dei listini prodotto. Attualmente si gestisce esternamente tramite file Excel.
- Mancanza di linee guida nella compilazione delle offerte. Si è avanzata l'ipotesi di un configuratore di prodotto che aiuti l'utente.
- Mancanza di gestione dei valori duplicati e controlli sulle foreign key porta a creare clienti e prodotti duplicati.

3.3.2 Richeste avanzate

Gestione ruoli e permessi

La scelta di coprire quanti più reparti possibili, ha fatto emergere la mancanza di gestione dei ruoli. Si è richiesto quindi di attribuire ad ogni utente un ruolo, a sua volta ognuno di essi avrà viste sul database e permessi dedicati. I ruoli individuati sono:

- Admin con visione sull'intero sistema, con possibilità esclusiva di modifica ad alcune tabelle fondamentali.
- Commerciali interni che possono gestire totalmente i clienti e contratti.
- Commerciali esterni all'azienda, esempio agenti a partita IVA, possono vedere solo i propri clienti e contratti.
- Amministrazione contabile, può modificare i dati contabili dei clienti. Questo ruolo può altresì esportare le voci contrattuali all'ERP per gestire la fiscalità.
- Project Manager con compito di revisionare tecnicamente il contratto. Questo compito è volto alla gestione a monte dell'invenduto.

- Visualizzatore senza permessi di scrittura.

Gestione dei contratti

Nella creazione dei contratti, core del progetto, si dovrà sviluppare il vero salto di qualità. Nelle interviste si è cercato di raccogliere le richieste che, anche dai primi rilasci, potessero dare al richiedente un sistema più ottimale. Tali richieste sono:

- Gestione storico dei dati in caso di modifica.
- Gestione di più versioni di uno stesso contratto legati dal medesimo codice progressivo. In caso di approvazione serve far decadere le versioni non approvate.
- Importazione dal PLM di dati tecnici dei prodotti. Questi dati sono fondamentali nei calcoli utili alla contrattazione (Es: Potenza elettrica totale consumata).
- Gestione degli stati di avanzamento di un progetto e relative notifiche.
- Revisione tecnica del venduto in modo da dare la possibilità di proporre all'occorrenza integrazioni contrattuali.

Capitolo 4

Analisi dei requisiti

4.1 Requisiti funzionali

Questa sezione è dedicata alla suddivisione dei dati raccolti in requisiti funzionali. La tabella, nella figura [4.1](#) seguente, suddivide i requisiti per area del progetto. Insieme al requisito e alla sua descrizione è presente anche l'elenco dei ruoli a cui esso è rivolto.

4.1. REQUISITI FUNZIONALI

Area / Modulo	Requisito Funzionale	Descrizione / Specifiche	Ruoli Coinvolti
Sicurezza & Accessi	Profilazione RBAC granulare	Gestione permessi per tabella (lettura/scrittura) tramite tabelle utenti e utenti_tipi.	Tutti i ruoli
Anagrafica Clienti	Controllo anti-duplicazione	Validazione obbligatoria dell'univocità tramite Partita IVA in fase di inserimento.	Amministrazione, Agenti, Commerciali
Anagrafica Clienti	Sedi multiple e tipizzazione	Gestione sedi legali/operative, destinazioni merce separate e distinzione cliente finale / rivenditore.	Amministrazione, Agenti
Anagrafica Clienti	Storico contratti cliente	Consultazione integrata di tutte le offerte e ordini emessi per lo specifico cliente.	Commerciali, Amministrazione
Catalogo Prodotti	Codifica univoca	Allineamento del codice prodotto tra Configuratore, Ad-Hoc ERP e Fusion PLM.	Admin, Commerciali
Catalogo Prodotti	Gestione obsoleti e visibilità	Blocco dei componenti obsoleti rilevati da PLM; flag di visibilità per migrazione progressiva da ERP.	Admin, PM / Engineering
Catalogo Prodotti	Selezione per Macrogruppi	Navigazione guidata ad albero (Macchina → Componenti → Accessori → Ricambi) con vincoli di compatibilità.	Commerciali Nuovi/Ricambi
Catalogo Prodotti	Scheda caratteristiche tecniche	Attributi tabellari strutturati: meccanici, elettrici, pneumatici, logistici e accessori correlati.	Commerciali, PM / Engineering
Catalogo Prodotti	Descrizioni standardizzate	Modello ibrido: voce principale da menu standardizzato (da Fusion) affiancata da campo note libero.	Commerciali, PM / Engineering
Offerte & Pricing	Struttura a Capitoli	Suddivisione delle offerte in capitoli gestiti come sottodistinte personalizzabili.	Commerciali
Offerte & Pricing	Pricing centralizzato e override	Prezzi sincronizzati da Ad-Hoc con possibilità di inserimento manuale per voci speciali o su misura.	Commerciali, Admin
Offerte & Pricing	Scontistica multi-livello	Calcolo sconti combinabili: per riga prodotto, per capitolo, globale (esclusa MDO), globale (inclusa MDO).	Commerciali
Offerte & Pricing	Calcoli tecnici ed economici	Calcolo automatico di potenza elettrica, consumo aria compressa, pesi, volumi e stima costi montaggio/trasporto.	Commerciali
Offerte & Pricing	Vincoli Incoterms e Paesi	Selezione assistita Incoterms (con vincoli UE/extra-UE); adattamento automatico a valuta, lingua e normative (es. UL-CSA).	Commerciali, Logistica
Offerte & Pricing	Alert informativi	Notifiche a video non bloccanti per segnalare scelte non conformi alle specifiche standard d'impianto.	Commerciali
Offerte & Pricing	Versioning e clonazione	Storico revisioni tracciato (Rev.01, Rev.02...) e funzione di duplicazione rapida di offerte esistenti.	Commerciali
Ordini & Workflow	Conversione e congelamento	Blocco del contratto commerciale alla conferma cliente e assegnazione codifica causale ordine.	Commerciali, Amministrazione
Ordini & Workflow	Notifiche multi-reparto	Alert push automatici inviati a Amministrazione, PM, Logistica e Produzione all'apertura dell'ordine.	Sistema (Cross-funzionale)
Ordini & Workflow	Esportazione verso ERP	Invio automatizzato dell'ordine confermato ad Ad-Hoc per avviare la fatturazione.	Amministrazione
Revisione Tecnica	Contratto tecnico parallelo	Gestione di una variante tecnica interna (senza valore legale) gestita dal PM con costificazione separata.	Project Management
Revisione Tecnica	Workflow decisionale modifiche	Bivio post-revisione: approvazione commerciale con nuovo contratto/RDA (impatto alto) o gestione interna (impatto basso).	PM, Commerciale
Stampe & Documenti	Layout di stampa per reparto	Generazione PDF personalizzati per Cliente, Amministrazione e Ufficio Tecnico.	Tutti i reparti
Localizzazione	Supporto multilingua	Interfaccia localizzata e traduzione automatica/manuale delle descrizioni contrattuali.	Commerciali, Agenti

Figura 4.1: Requisiti funzionali raccolti.

4.2 Requisiti non funzionali

Questa sezione, a differenza della precedente raccoglie i requisiti non funzionali. La tabella che seguirà, nella figura 4.2, insieme al requisito stesso, raccoglie anche la sua categoria e la sua priorità all'interno dell'implementazione. I requisiti sono ordinati per priorità.

Categoria	Requisito Non Funzionale	Priorità	Specifica Tecnica / Obiettivo
Affidabilità	Eliminazione crash e blocchi	Critica	Stabilità operativa durante inserimento massivo righe, calcoli e stampa, azzerando i blocchi tipici di Access.
Interoperabilità	Connessione nativa ERP	Critica	Integrazione bidirezionale diretta via query SQL Server con Ad-Hoc (anagrafiche, listini ed export ordini).
Disponibilità	Architettura offline (LocalDB)	Alta	Continuità operativa garantita anche in assenza di rete o VPN tramite istanza locale SQL Server LocalDB.
Integrità Dati	Sincronizzazione automatica	Alta	Motore di sync bidirezionale tra LocalDB e SQL Server aziendale con logica di risoluzione conflitti.
Resilienza	Backup automatici in VPN	Alta	Salvataggio automatico del database locale durante le connessioni VPN per prevenire perdite o corruzioni dati.
Integrità Dati	Unificazione codici prodotto	Alta	Coerenza assoluta delle codifiche articolo tra Configuratore, Ad-Hoc ERP e Fusion PLM senza disallineamenti.
Usabilità	Workflow guidati (Wizard)	Alta	Percorsi passo-passo che guidano la compilazione per azzerare omissioni o errori di configurazione commerciale.
Usabilità	Viste orientate al ruolo (RBAC)	Alta	Maschere e sezioni della UI mostrate dinamicamente in base alle prerogative del ruolo utente loggato.
Portabilità	Framework moderno	Media	Architettura per ambiente desktop con predisposizione a target mobile.
Prestazioni	Calcolo istantaneo in-memory	Media	Elaborazione immediata a runtime di formule per potenze, consumi d'aria, pesi, volumi e stime logistiche.
Qualità Processo	Area di staging ordini	Media	Area di transito temporanea per la revisione amministrativa delle conferme d'ordine prima del caricamento contabile.
Evolutività	Predisposizione CAD / PLM	Bassa	Architettura aperta al recupero di disegni tecnici e visualizzazione di modelli 3D direttamente da Fusion.

Figura 4.2: Requisiti non funzionali raccolti.

Capitolo 5

Progettazione dell'architettura

5.1 Premessa sulle scelte architetturali

Le scelte architetturali sono dipese in primo luogo dall'infrastruttura aziendale, e successivamente dalle richieste degli utenti durante gli incontri. L'utilizzo di SQLServer, come motore di database, è stato adottato come base di tutte le scelte successive. Da questo caposaldo, la ricerca si è indirizzata verso un framework che interagisse sia con SQLServer sia con SQLite.[SQL]. La necessità di operare offline ha guidato la ricerca di un sistema installabile in locale, escludendo così le soluzioni web based. La scelta è quindi ricaduta sulla piattaforma .NET[.NE] con il linguaggio C# che, in quanto parte dell'ecosistema Microsoft, è perfettamente integrata con i database prima citati. La piattaforma .NET è espandibile mediante l'utilizzo di NuGet[NuG], che permette di aggiungere package e quindi funzionalità al progetto. I pacchetti sono sviluppati sia da Microsoft sia da terze parti, e vengono aggiornati e supportati continuamente. Questa manutenzione continua evita di incorrere in debiti tecnici futuri. L'ambiente .NET è nativamente multiplatforma e questa proprietà soddisfa le necessità dei richiedenti. Secondo le linee guida Microsoft, per lo sviluppo delle view multiplatforma, è consigliato l'utilizzo del package NuGet .NET MAUI[MAU]. In fase di compilazione, MAUI converte il codice sorgente in codice nativo per la piattaforma di destinazione. MAUI, integrato con il package NuGet "Community Toolkit Maui"[Opea], consente di costruire le interfacce grafiche in codice C#. Nativamente MAUI supporta solo il linguaggio

Markup XAML[Micf].

5.2 Deployment

La scelta degli ambienti di sviluppo precedentemente citati porta a dover considerare il deployment dell'applicativo. Il software verrà installato in locale su ogni dispositivo, sia esso desktop o mobile, e si conatterà al database centrale. All'interno dell'azienda è presente un dominio Active Directory (AD), dove ogni oggetto del dominio ha un Internet Protocol (IP) statico. Questa configurazione permette la connessione al database, risiedente in un server all'interno del dominio, in modo agile tramite IP, se si è connessi all'interno della Local Area Network (LAN). Lo scambio di dati tra il frontend e il backend avviene mediante protocollo Transmission Control Protocol/Internet Protocol (TCP/IP). Si è valutato, per chi accede dall'esterno del dominio, di utilizzare la rete VPN, stabilendo una connessione virtuale che permette di essere all'interno della LAN. Il database locale, in SQLite, è strutturato come una copia del database centrale. I dati vengono allineati tra i due database mediante un processo di sincronizzazione. In assenza di connessione, verificabile tramite ping, si accede al database locale ed, una volta ristabilita la connessione, i dati vengono sincronizzati. Il diagramma dei deployment è riassunto in figura 5.1.

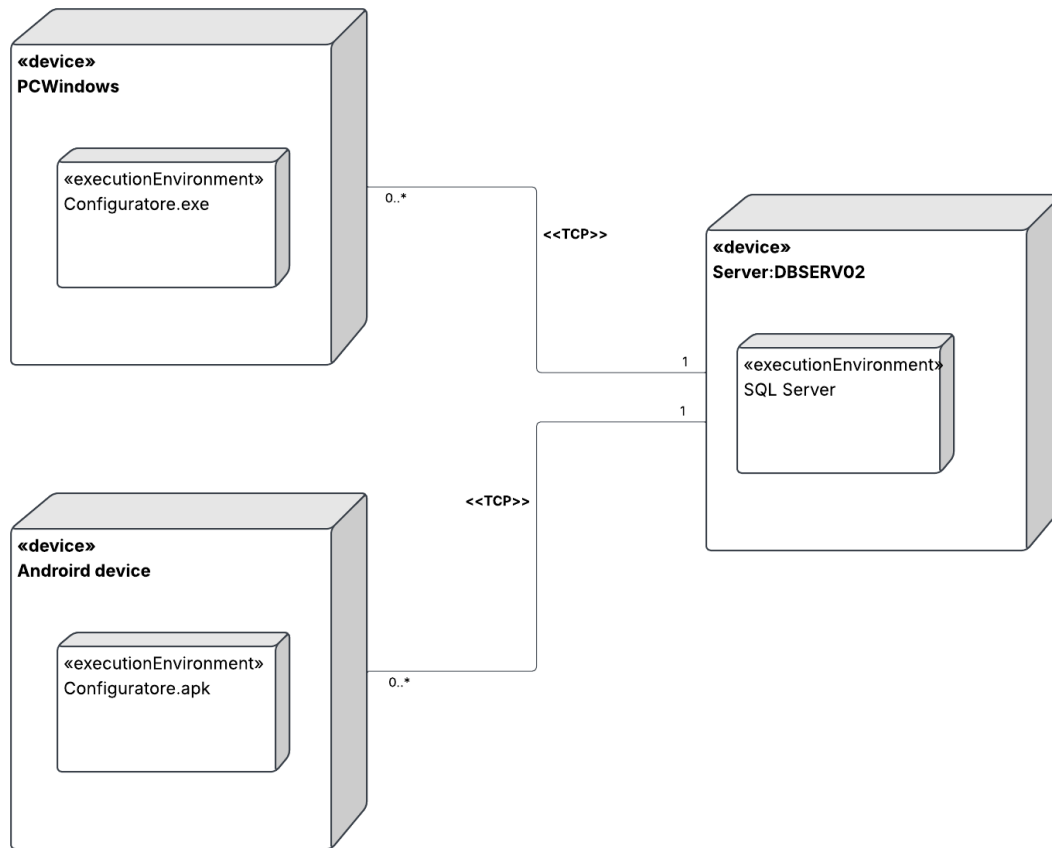


Figura 5.1: Diagramma dei deployment

Nella fase di sviluppo, per testare la struttura del database, è stato scelto di utilizzare una versione di SQLServer locale, denominata SQL Express[Mice]. SQL Express condivide con SQLServer tutte le caratteristiche principali. Questa proprietà ha permesso di costruire query e testare funzionalità localmente in modo compatibile con il database di produzione.

5.3 Package

L'intero software è stato suddiviso in tre distinti package, con responsabilità diverse. Si è cercato di seguire il principio Don't Repeat Yourself (DRY) e la riusabilità

degli stessi.

5.3.1 Interfaccia al database

Il package di interfaccia al database, denominato “DatabaseManagementPackage”, ha il compito di gestire tutti gli aspetti legati a quest’ultimo, come:

- gestione delle connessioni ai vari database, come SQLite e SQLServer o nel caso dei test, SQL Express;
- gestione delle transazioni;
- gestione dei permessi utente, bloccando le operazioni non consentite;
- conversione, mediante Object-Relational Mapping (ORM), delle tabelle del database in entity;
- mappatura di entity in file neutri, proteggendo il database;
- gestione di Repository per ogni entità, che espongono le operazioni Create Read Update Delete (CRUD);
- gestione di una suite di test per verificare le operazioni precedenti. Questa suite è risultata fondamentale nell’evoluzione del database, validando i cambiamenti apportati;
- esporre al di fuori del package solo file neutri e la classe manager del database.

Nell’implementazione si è utilizzato il pacchetto NuGet Entity Framework Core (EF Core)[[Micc](#)]. Questo modulo permette una gestione efficiente delle operazioni di accesso ai dati, consentendo di astrarre le tabelle del database in classi di entity. Le entity sono state a loro volta mappate in Data Transfer Object (DTO), proteggendo i dati. Questa seconda mappatura è stata realizzata mediante il package “Mapster”[[Opeb](#)], appartenente a terze parti. La mappatura entity-DTO è stata realizzata in modo bidirezionale in quanto, in alcuni casi, è necessario convertire un DTO in entity per poterlo salvare nel database.

5.3.2 Libreria di componenti view

La creazione di interfacce grafiche è stata supportata da un package dedicato, che racchiude in sé factory di componenti, costanti e utilità. Il package, denominato “ViewComponentLibrary”, è stato concepito a sé stante per favorire un suo impiego in implementazioni future. La libreria si appoggia a sua volta al package NuGet “Community Toolkit Maui”[Opea]. Questo package permette di scrivere le interfacce grafiche in codice C# Markup, evitando quindi l’utilizzo del linguaggio XAML[Micf], standard di sviluppo per le interfacce MAUI.

5.3.3 Modulo main e view

Il package principale del progetto è dedicato alla gestione dei componenti e dei flussi di lavoro. Il package, denominato “ConfiguratoreCommerciale”, ha numerosi compiti, tra cui la gestione della Dependency Injection, la creazione delle view e dei rispettivi ViewModel. In questo package risiedono infatti i ViewModel, che si interpongono tra le view e i repository, esponendo i dati tramite proprietà osservabili (Observable Properties). La Dependency Injection, invece, permette di iniettare le dipendenze necessarie alle classi, come ad esempio i repository, le classi di logging o le view stesse se registrate. Le classi di view, utilizzando i componenti della libreria, vengono realizzate tenendo conto del dispositivo di destinazione. La differenziazione è permessa dall’utilizzo della compilazione condizionale, distinguendo il codice da compilare in base alla piattaforma di destinazione. Un esempio di tale condizione è il seguente:

```
1 #if ANDROID
2     // codice per piattaforma Android
3 #elseif WINDOWS
4     // codice per piattaforma Windows
5 #endif
```

Questa possibilità permette anche di eseguire parti di codice solo in fase di DEBUG. Gli aspetti generali gestiti dal package possono essere riassunti come segue:

- creazione delle interfacce grafiche;
- traduzione a compile time del codice C# in XAML;

- traduzione a compile time in elementi nativi per la piattaforma di destinazione;
- gestione delle interfacce mediante ViewModel, che a loro volta tramite proprietà osservabili modificano la view in risposta ad eventi;
- gestione della navigazione tra le interfacce grafiche;
- gestione dei permessi, mostrando solo i comandi che competono al ruolo dell'utente;
- gestione della Dependency Injection mediante la classe *MauiProgram.cs*;
- creazione, in fase di avvio, di una cache in Random Access Memory (RAM) di dati immutabili durante la sessione, come tabelle di lookup.

Il diagramma dei package è riassunto in figura 5.2.

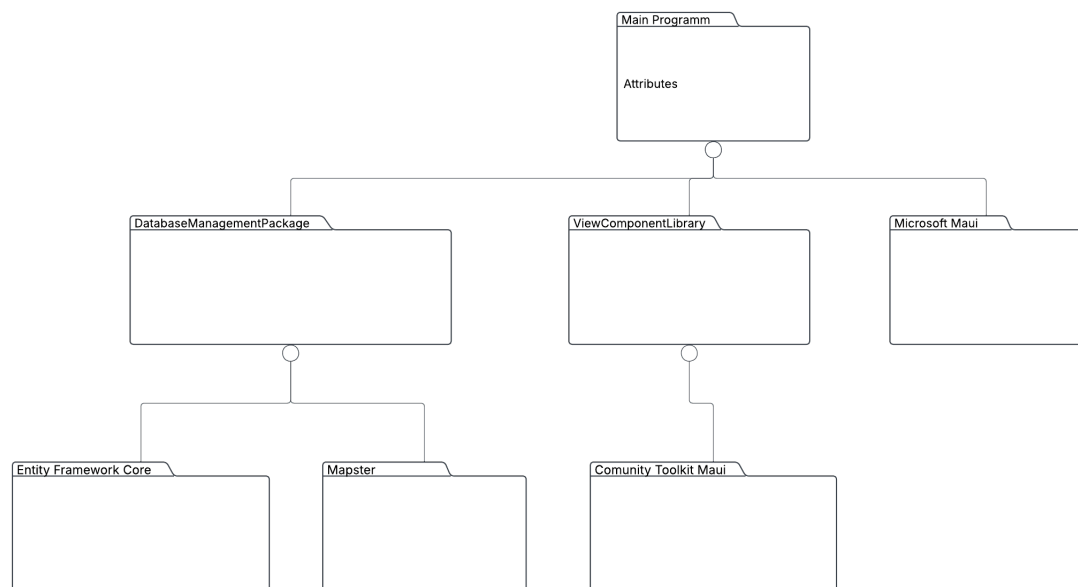


Figura 5.2: Diagramma dei package

5.4 Flussi di lavoro

I flussi di lavoro citati in figura 2.2 sono risultati complessi e poco efficienti. Si è quindi deciso di inserire una revisione tecnica, da parte di un utente specializzato, degli impianti venduti. Il nuovo diagramma, in figura 5.3, mostra un flusso più lineare che porta a valle meno errori. Determinare i nuovi flussi lavorativi è fondamentale per progettare gli automatismi a cui sono soggette le notifiche intra-reparto.

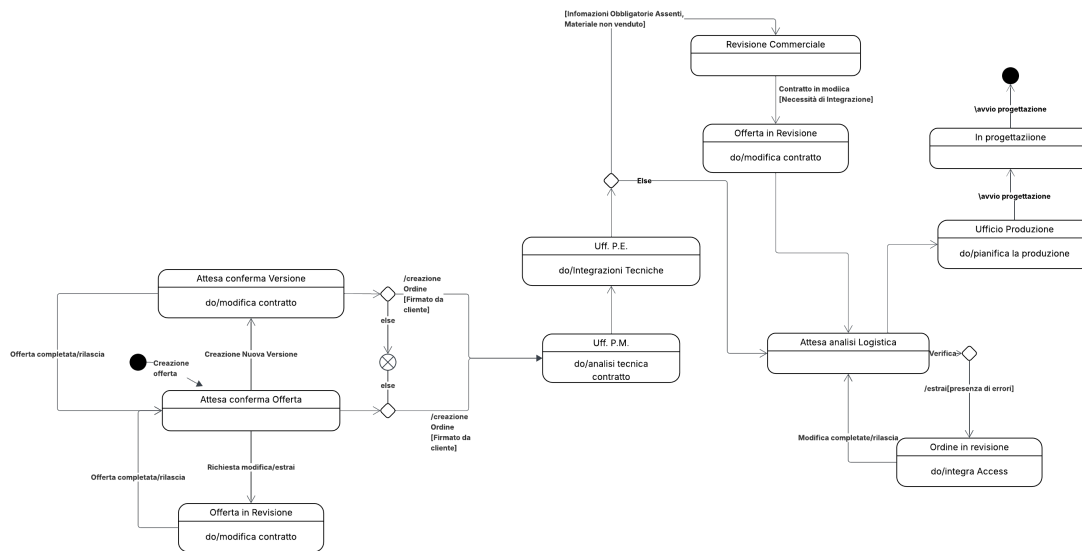


Figura 5.3: Nuovo diagramma degli stati di un contratto

Capitolo 6

Scelte progettuali

6.1 Scelta modello di sviluppo

Il modello di sviluppo scelto è basato sulla necessità di prototipare e testare con gli utenti le funzioni nell'arco dell'intero sviluppo. Si è quindi deciso di adottare un modello agile, supportato dalla creazione di test delle funzioni e dei prototipi delle interfacce. La progettazione è stata iterativa e incrementale, seguendo i canoni dell'agilità. Tutti i metodi di interazione col database sono stati testati da appositi Unit Test che ne validassero il comportamento. I test di regressione hanno permesso di convalidare le modifiche apportate al database e di conseguenza al software. Le interfacce grafiche, al contrario, non sono state supportate da test, ma prototipate e approvate da utenti scelti. La rifattorizzazione del codice è stata affrontata con regolarità seguendo il principio DRY. Nel seguire tale principio, si sono create più classi di Factory e classi di utilità che unificassero quanto più possibile il codice comune.

6.2 Suddivisione e contenuto dei rilasci

Le tempistiche dei rilasci sono state decise insieme al Project Manager aziendale. Il collega è stato di supporto nella stima dei tempi di sviluppo e nella conversione del database esistente, guidando lo sviluppo attraverso le tecniche professionali. Nel primo rilascio, si è scelto di includere tutti i requisiti già presenti nel precedente

applicativo. In tale rilascio, con consegna stimata per gennaio 2027, sono presenti le seguenti funzionalità:

- gestione utenti e permessi;
- anagrafica clienti;
- creazione delle offerte;
- versioning offerte;
- creazione ordini;
- import ed export dati, da e verso altri database;
- creazione reportistica;
- gestione multilingua dell'interfaccia applicativa;
- prototipo mobile;
- modulo di configurazione prodotto.

Si è scelto di non includere la funzionalità offline, in quanto essa necessita di un database centrale stabile. La funzionalità offline verrà implementata nel secondo rilascio. Questa fase avverrà una volta che il software sarà in produzione, per poter contare su un database pressoché stabile. Nel secondo rilascio saranno quindi presenti:

- gestione revisione tecnica e differenze con contratto firmato;
- inserimento di calcoli automatici sulla base delle caratteristiche del venduto (es. potenza elettrica totale assorbita);
- gestione multi-valuta per ordini internazionali;
- apertura dei disegni PDF dal server del PLM;
- report avanzati;
- implementazione database locale con logiche di sincronizzazione.

6.3 Scelta framework ORM

All'interno della sezione sull'architettura, è stata citato la scelta del framework EF Core 5. Questo framework ha permesso di mappare le tabelle del database in classi mediante la tecnica ORM. Questa tecnica consiste nel mappare ogni tabella del database in una classe che ne rappresenti l'entità. I legami tra le entità sono gestiti mediante la classe **Context**. L'ORM ha permesso, con l'utilizzo di script PowerShell, di ricreare in modo rapido tutte le entity e il context a fronte di cambiamenti del database. Tale approccio può essere definito database-first, in quanto la progettazione avviene principalmente sul database e il software ne eredita le modifiche. Le queries sono state scritte tramite le funzioni Language INtegrated Query (LINQ), che hanno permesso di formularle in linguaggio C#.

Listing 6.1: Esempio di query LINQ

```
1 public class Example{
2     public async Task<ClientiDestinazioniDTO> GetLocationDetailsById(int?
      idLocation)
3     {
4         ClientiDestinazioni? location = await _db.ClientiDestinazioni
5             .Where(l => l.CdeId == idLocation)
6             .Include(l => l.CdeNazioneNavigation)
7             .Include(l => l.CdeTipoNavigation)
8             .Include(l => l.CdeClienteNavigation)
9             .Include(l => l.CdeNormaElettricaNavigation)
10            .Include(l => l.CdeZonaNavigation)
11            .AsNoTracking()
12            .FirstOrDefaultAsync();
13        return _mapper.Map<ClientiDestinazioniDTO>(location!);
14    }
15 }
```

In ottica di offline, le queries in LINQ sono compatibili con il database locale SQLite se si limitano le funzioni utilizzate a quelle supportate da esso. Le funzioni compatibili sono ad esempio: *Where*, *Select*, *OrderBy*, *GroupBy*, *Join*, *Distinct*.

Uno degli aspetti negativi di LINQ è la lentezza con grandi volumi di dati. Lo svantaggio è dovuto alla traduzione del codice in query compatibili con SQL. Si è valutato però, in caso si presentino problemi di performance, di ottimizzare il database mediante indici in prima battuta e, successivamente, ottimizzando le singole query. EF Core permette di ottimizzare le queries scrivendole in SQL puro, bypassando la traduzione da LINQ.

6.4 Scelta del framework grafico

Il framework grafico scelto aveva come obiettivo soddisfare il requisito della portabilità e dell'offline. Le scelte vagliate sono state:

- frontend web based, con l'utilizzo di HTML, CSS e JavaScript e interrogazione del database centrale mediante Application Programming Interface (API);
- utilizzo del framework MAUI, nativamente cross-platform e integrato perfettamente con il backend.

L'opzione web based, con interfacce facilmente adattabili a qualsiasi dispositivo, ha presentato due svantaggi principali. La difficoltà principale risiedeva nell'utilizzo in assenza di connessione. In questo scenario, per funzionare, richiedeva che i file e il database fossero presenti sul dispositivo. Questa caratteristica rende difficoltoso gestire gli aggiornamenti. Il secondo motivo, non trascurabile per le tempistiche, è l'utilizzo di più linguaggi di programmazione. Quest'ultimo non massimizza l'esperienza acquisita con l'implementazione degli altri moduli. Il framework MAUI di .NET sopprime a entrambi i problemi. Il programma è fisicamente installato sui dispositivi e il loro aggiornamento può essere gestito centralmente all'interno di uno store Microsoft. Essendo basato su .NET e avvalendosi del framework "Community Toolkit Maui", le interfacce sono scritte in C#.

6.5 Scelta del pattern di progettazione

Insieme a MAUI, Microsoft suggerisce l'utilizzo del pattern Model View ViewModel (MVVM). Il modello MVVM consente di separare in modo pulito le logiche di business e le relative interfacce utente. La gestione separata risolve numerosi problemi, quali la possibilità di ridisegnare le interfacce utente senza dover modificare le classi che le governano. Un secondo vantaggio è la possibilità di consentire agli sviluppatori e ai progettisti dell'interfaccia utente di collaborare più facilmente durante lo sviluppo delle rispettive parti di un'app. [\[Micb\]](#) Il modello MVVM è composto da tre elementi fondamentali che interagiscono come in figura [6.1](#):

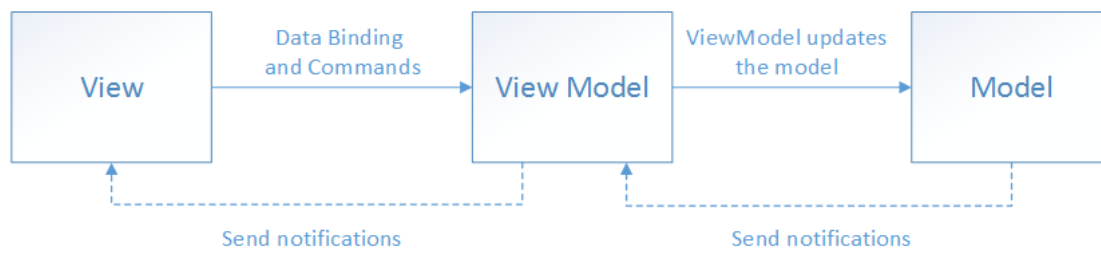


Figura 6.1: Diagramma funzionamento MVVM

Questo modello, separando la logica della view da quella di business, ha permesso di prototipare le interfacce nei primi rilasci e in futuro riprogettarle migliorando l'accessibilità e l'User Experience (UX).

Capitolo 7

Progettazione del database

7.1 Esportazione vecchio database

La creazione della nuova base di dati è partita dall'esportazione e dallo studio dell'attuale database Access. L'esportazione è avvenuta per mezzo di SQL Server Migration Assistant (SSMA), che ha permesso di esportare solo la struttura esistente in SQL Server. Una volta ricostruita la struttura, essa ha subito un processo di ristrutturazione che ha portato ad avere un database più leggibile, manutenibile e performante.

7.2 Denominazione tabelle e campi

Il primo processo di ristrutturazione è stato la creazione delle regole di denominazione per le tabelle e le colonne. Le regole adottate sono state:

- Campi delle tabelle in minuscolo, separati da underscore, ad esempio: *nome_campo*.
- A ogni campo viene apportato il prefisso di appartenenza alla tabella, ad esempio: *prefTabella_nome_campo*. Questa modifica rende ogni campo univoco e facilmente identificabile in caso di join tra tabelle.
- Le chiavi primarie di ogni tabella sono degli indici progressivi, rendendo unica ogni riga. Il campo segue il pattern *prefTabella_id*.

Nelle tabelle sono stati aggiunti dei campi di audit, come la data e l'utente di creazione, ma anche la data e l'utente di modifica. Infine, sempre come campo comune a tutte le tabelle, è stato aggiunto il campo boolean *deleted* per la gestione della cancellazione logica. Questi campi comuni sono stati pensati per essere di supporto nella gestione dei dati, permettendo la gestione dei backup e delle modifiche senza cancellare mai fisicamente i dati.

7.3 Utilizzo di SSIS nel processo di migrazione

I successivi processi di modifica e popolamento del database sono stati gestiti tramite SQL Server Integration Services (SSIS). Questo strumento permette la creazione, anche mediante interfaccia grafica, di pacchetti di query eseguiti secondo un flusso preimpostato. Si è quindi creato un primo flusso di query che in ordine:

1. Crea se non esiste il database.
2. Se presenti le tabelle, le elimina.
3. Crea le tabelle seguendo un ordine consono alle foreign key.
4. Popola le tabelle di lookup.
5. Popola le tabelle con dati del vecchio database.
6. Popola le tabelle restanti con dati di test.

Questo flusso ha permesso una continua ristrutturazione del database, permettendo di testare le modifiche con facilità. Nelle fasi più avanzate, al flusso prima citato sono state aggiunte le operazioni per la gestione dei backup, e per la gestione della sincronizzazione dei dati provenienti da altri database.

7.4 Ristrutturazione database

7.4.1 Eliminazione ridondanze e normalizzazione

La fase di studio dell'attuale database ha portato alla luce diverse ridondanze e dati non normalizzati. All'interno erano presenti più tabelle con lo stesso contesto

di utilizzo o tabelle non utilizzate. Si è quindi proceduto all'eliminazione di tali tabelle ed alla normalizzazione delle restanti in seconda forma normale. Un'altra modifica sostanziale è stata l'unificazione di tabelle simili in un'unica tabella, discriminate mediante campo tipo proveniente da un lookup. Un esempio di questo contesto sono le tabelle *Offerta* e *Ordine*, dove una è la diretta evoluzione della precedente. Le due tabelle sono state unite sotto il cappello della tabella *Documenti*, discriminata da un campo lookup che ne identifica il tipo. In queste tabelle in particolare era importante tenere traccia del legame PadreFiglio, si è aggiunto quindi un campo `doc_documento_padre` che, mediante chiave importata, permette mediante auto join di ricostruire il legame. Il metodo così pensato, in ottica di ampliamento, permette la gestione di ulteriori tipologie di documenti senza cambiare la struttura.

7.4.2 Creazione tabelle di lookup

L'estendibilità del database mediante le tipologie associate alle tabelle passa dalla creazione di tabelle di lookup. Queste tabelle sono considerate fisse, e non possono essere modificate dall'utente finale, ma solo da un amministratore tramite query. Tali tabelle, con al più un campo descrittivo, rendono la gestione dei dati più semplice, potendo filtrare su di essi. Un vantaggio, a livello di software, è il poterle richiamare internamente mediante classi di enumerazione, rendendo il codice più leggibile e manutenibile.

7.4.3 Aggiunta tabelle non presenti

Gestione dei prodotti

Il database originale gestiva i prodotti come oggetti atomici, senza la possibilità di gestire i loro componenti. Si è quindi deciso di gestire le distinte; si è posta la tabella *Prodotti* in auto join con cardinalità n ad n . Tale relazione a livello di schema logico è stata realizzata mediante la tabella *Prodotti_composizioni*. La tabella in questione aggiunge, oltre alle chiavi dei prodotti, anche il quantitativo della riga stessa.[7.1](#) La gestione così pensata permette di effettuare ricerche approfondite sui ricambi di un determinato prodotto. Questo sistema, insieme ad appositi lookup,

ha inoltre permesso la costruzione del modulo Configure, Price, Quote (CPQ) per la configurazione di prodotto.??.

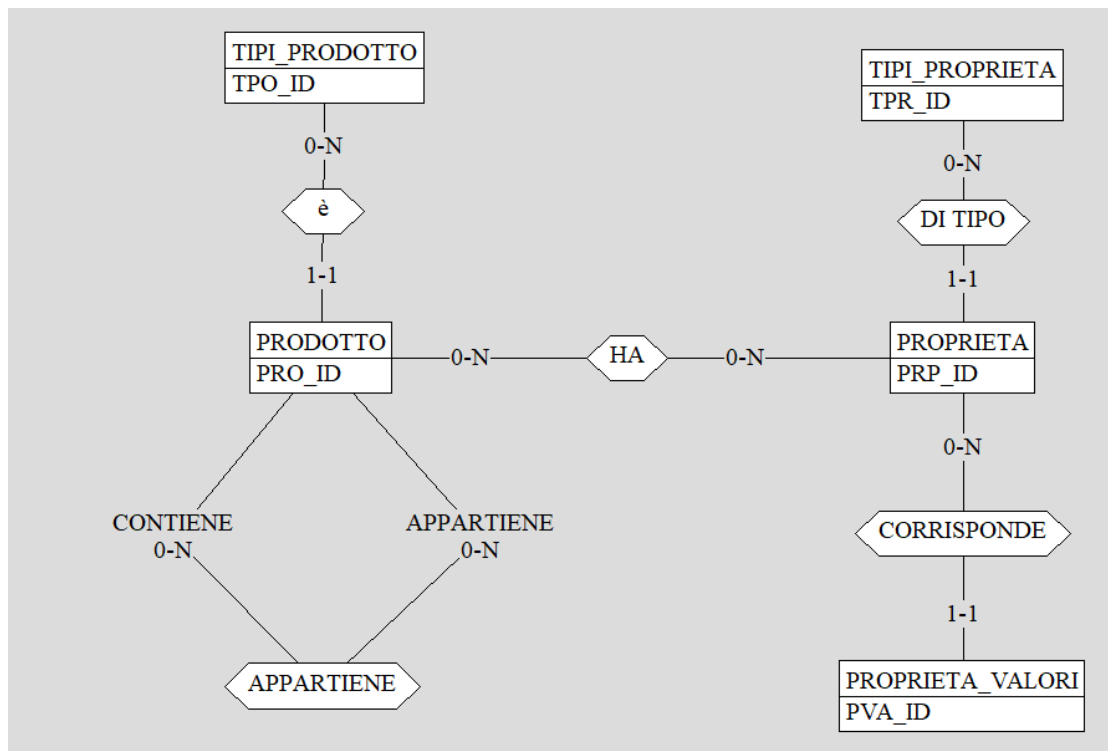


Figura 7.1: Schema ER dei prodotti

Tabelle gestione lingue

Si è deciso di aggiungere delle apposite tabelle per la gestione delle traduzioni di campi del database. Le tabelle predisposte contengono al loro interno la chiave della tabella di riferimento, il campo da tradurre e la chiave importata dalla tabella delle *lingue*. Quest'ultima tabella non era presente nella soluzione di partenza. La gestione così pensata permette di estendere le traduzioni a più lingue senza modificare la struttura del database. Si è anche aggiunta una apposita tabella per gestire le traduzioni nelle reportistiche dei documenti. Questa tabella mappa le tipologie di reportistica richieste dal cliente con le apposite lingue concordate. Capita infatti di avere ad esempio manuali in lingua inglese, ma etichette sui

macchinari con la lingua del luogo. Il diagramma ER delle lingue è visibile in figura 7.2.

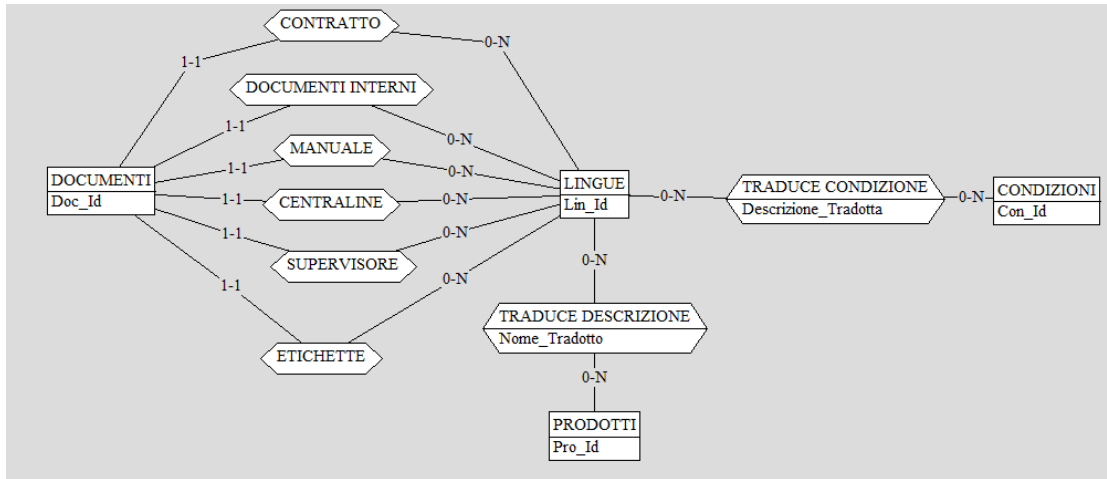


Figura 7.2: Schema ER delle lingue

Creazione tabelle di backup

Si è scelto di gestire solo un ristretto numero di essenziali tabelle, come ad esempio le tabelle dei documenti e dei prodotti, delle righe dei documenti o dei prodotti. La gestione non è stata capillare su tutte le tabelle, in quanto si sono scelte solo le tabelle più centrali del dominio. Questo approccio è volto a evitare di appesantire il database con tabelle, dati e trigger non necessari. Le strutture delle tabelle di backup rispecchiano quelle delle tabelle originali, ma senza vincoli né chiavi primarie o secondarie, garantendo così la possibilità di salvare più modifiche della stessa tupla. Il popolamento avviene mediante dei trigger, essi scattano ad ogni operazione di Update eseguita sulla tabella originale. Tramite query ed il campo di audit *data_modifica* si è quindi in grado di ricostruire lo storico dei dati e di ripristinare eventuali errori. Il trigger è marcato come “AFTER UPDATE”. In combinazione con il dato temporaneo “deleted” permette di salvare il dato prima della modifica, garantendo la possibilità di ripristinare lo stato precedente.

Listing 7.1: Esempio di trigger

```
1 CREATE TRIGGER trg_prodotti_update
2 ON [dbo].[prodotti]
3 AFTER UPDATE
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     INSERT INTO [dbo].[prodotti_backup] (
9         [pro_id], [pro_descrizione], [pro_codice], [pro_prezzo],
10        [pro_potenza_installata], [pro_aria_compressa], [pro_peso_a_vuoto],
11        [pro_volume], [pro_flag_ricambio], [pro_flag_obsoleto],
12        [pro_tipo], [pro_tipo_merceologia], [pro_note],
13        [synced_on], [creation_date], [creation_user],
14        [last_edit_date], [last_edit_user], [deleted]
15    )
16    SELECT
17        [pro_id], [pro_descrizione], [pro_codice], [pro_prezzo],
18        [pro_potenza_installata], [pro_aria_compressa], [pro_peso_a_vuoto],
19        [pro_volume], [pro_flag_ricambio], [pro_flag_obsoleto],
20        [pro_tipo], [pro_tipo_merceologia], [pro_note],
21        [synced_on], [creation_date], [creation_user],
22        [last_edit_date], [last_edit_user], [deleted]
23    FROM deleted;
24 END;
```

Tabelle di utilità

Nel database sono state incluse delle tabelle di utilità, che non hanno un legame diretto con le altre tabelle, ma saranno utilizzate per la gestione e manutenzione del database. Queste tabelle sono:

- *Tabelle*, questa tabella mappa per ogni tabella del database il livello di ruolo necessario per ogni operazione CRUD. Tramite apposito lookup, a ogni utente è associato un ruolo, corrispondente al livello in ordine decrescente per importanza. Si ha quindi 0 per l'Admin e il 6 per il visualizzatore.
- *Parametri*, questa tabella contiene solo la versione del database. Questo dato permette di verificare la presenza di aggiornamenti del database locale prima dell'avvio del sistema.
- *Query*, questa tabella raccoglie le query necessarie per aggiornare la struttura del database locale con quella in remoto.

7.4.4 Importazione dei dati e Stored Procedure

Come da analisi, è stato richiesto di prelevare e sincronizzare alcuni dati da altri database. Prima della costruzione delle vere e proprie query di sincronizzazione, si sono studiate le tabelle di provenienza per ricostruire un ambiente simile, facilitando la costruzione delle query. In particolare, dal database Fusion, si è ricostruita la struttura di assegnazione dinamica delle proprietà a un prodotto. Tale struttura, ponendo la cardinalità n ad n tra prodotti e proprietà, permette l'assegnazione e la creazione di nuove proprietà senza modificare la struttura del database. Alla tabella Proprietà è associata la tabella *Proprietà_valori* per la gestione dei valori standard. Una volta ricostruite le strutture si sono create le seguenti Stored Procedure:

- Da ERP, importazione e aggiornamento delle anagrafiche dei prodotti.
- Da ERP, importazione delle distinte dei prodotti.
- Da PLM, importazione e aggiornamento delle proprietà dei prodotti.

Tali stored procedure hanno al loro interno dei controlli che agiscono solo quando si verifica una discrepanza. Oltre a ciò saranno da prevedere altre Stored Procedure di importazione per la gestione di altri dati, tali dati sono ad esempio i clienti, tassi di cambio, ecc. Una volta in produzione verranno schedate giornalmente garantendo la sincronizzazione dei dati.

7.4.5 Schema logico

Lo schema logico risultante è visibile in figura [7.3](#).

Capitolo 8

Creazione Model

8.1 Mapping delle classi

Il model dell'applicativo è suddiviso in parte nel package “DatabaseManagement-Package” e in parte, per le parti non legate al database, nel package principale. Il package di interfaccia al database è stato realizzato partendo, mediante EF Core, dalle tabelle del database, generando le classi entity. Una volta generate le classi entity, si sono individuati gli oggetti fondamentali del dominio, che sono stati mappati in classi DTO mediante il package “Mapster”. Questi oggetti sono stati scelti in quanto rappresentano le entità principali del dominio, come ad esempio le entità “Cliente”, “Prodotto” e “Documento”. Le classi DTO sono state realizzate partendo dalla struttura delle entity e, in corrispondenza dei campi delle entity legati alle foreign key, sono stati adottati due approcci differenti. Se il campo aveva una corrispondenza con una DTO, si richiama quest'ultima; altrimenti si è deciso di mappare il campo *Dictionary<int,string>*, dove si salvavano l'ID e la descrizione. Il mapping è bidirezionale per poter riconvertire, quando necessario, un DTO in entity per poterlo inserire nel database.

8.2 Struttura del package

Il package è stato strutturato in modo da separare le classi di test, i repository e le classi e interfacce esposte. All'esterno del package oltre alle classi DTO ed

Enum relativi ad alcuni Lookup, è stata esposta una classe univoca che espone i vari repository, denominata “DatabaseManager”. Questa soluzione permette di avere un unico punto di accesso ai repository, evitando di esporre le classi interne del package. Il vantaggio della classe singola si ha anche nella gestione della Dependency Injection, in quanto si può registrare un’unica classe per tutti i repository, evitando di registrare ogni singolo repository. Si è poi realizzata una classe di test per ogni repository, che ne testasse le funzionalità di base. Nel package è stata inoltre predisposta una classe con compito di mantenere salvati i dati dell’utente loggato, come ad esempio l’ID dell’utente, il nome e cognome, il ruolo e i permessi. Questa classe, registrata come singleton, permette di avere un unico punto di accesso ai dati dell’utente loggato, permettendo la gestione dei permessi in ogni parte dell’applicativo.

8.2.1 Classi di Test

Le classi di test realizzate sono state pensate con l’obiettivo di funzionare anche in presenza di dati reali. Per realizzare ciò si sono creati prima di ogni test dei savepoint del database, che permettessero di tornare allo stato iniziale. Successivamente, per ogni test, si creavano i dati di test fittizi che potessero validare le funzionalità del repository senza il rischio di prelevare dati reali. Questo approccio ha richiesto molto tempo per la ricostruzione dei test più complessi, ma ha permesso di testare piccoli cambiamenti senza dover ripristinare il database. A fronte del tempo impiegato prima si è utilizzata Artificial Intelligence (AI) generativa per creare le bozze dei test, che successivamente sono state modificate e, in alcuni casi, riscritte completamente, per adattarle al contesto reale. Per realizzare ciò si è adottato un approccio articolato come segue:

1. Si sono scritti i metodi del repository.
2. Si sono generati i test sulla base dei metodi del repository.
3. Si sono corretti i test per renderli accurati e validi.
4. Si sono corretti i metodi del repository sulla base del risultato dei test.

Questo approccio ha permesso di avere un ciclo di sviluppo più veloce, riducendo il tempo di scrittura dei test.

8.2.2 Repository

I repository sono stati divisi in base alle entità principali del dominio, come ad esempio i Prodotti. Ogni repository poteva a sua volta gestire più DTO, come ad esempio *ProdottoDTO* e *ProdottocomposizioneDTO* appartenenti allo stesso oggetto di dominio. Oltre alle operazioni CRUD, si è scelto per quanto riguarda le operazioni di ricerca, di utilizzare dei metodi che permettessero di inserire molteplici criteri di ricerca. All'interno sono poi stati utilizzati i Queryable per costruire la query in esame.

Listing 8.1: Esempio di query Queryable

```
1 public class Example{
2     public async Task<List<UtentiDTO>> GetFilteredUsers(string? username =
3         null,
4         string? name = null, string? surname = null, string? email = null, int
5         ? userId = null, bool? deleted = false,
6         CancellationToken cancellationToken = default)
7     {
8         IQueryable<Utenti> query = db.Utenti
9             .Where(u => u.Deleted == deleted)
10            .Include(u => u.UteTipoNavigation)
11            .AsNoTracking();
12
13         if (!string.IsNullOrEmpty(username))
14         {
15             query = query.Where(u => u.UteUsername!.Contains(username));
16         }
17         if (!string.IsNullOrEmpty(name))
18         {
19             query = query.Where(u => u.UteNome!.Contains(name));
20         }
21         if (!string.IsNullOrEmpty(surname))
22         {
23             query = query.Where(u => u.UteCognome!.Contains(surname));
24         }
25         if (!string.IsNullOrEmpty(email))
26         {
27             query = query.Where(u => u.UteEmail!.Contains(email));
28         }
29
30         if (userId.HasValue)
```

```
29         {
30             query = query.Where(u => u.UteTipo == userTypeId.Value);
31         }
32
33         List<Utenti> entities = await query
34             .OrderBy(u => u.UteNome)
35             .ToListAsync(cancellationToken);
36
37         return _mapper.Map<List<UtentiDTO>>(entities);
38     }
39 }
```

Questo approccio ha permesso di costruire query complesse, senza dover scrivere molteplici metodi di ricerca. Un'altra scelta che ha permesso di ridurre il codice ripetitivo è stata quella di rimappare le nuove entità, create in DTO dalle classi di model del package principale, in classi di entity prive di Id che fossero pronte ad essere inserite nel database. Tutti i metodi all'interno dei repository sono stati realizzati come asincroni, per evitare di bloccare il thread principale dell'applicativo e, in particolare, l'interfaccia grafica.

8.3 Gestione dei dati e delle transazioni

Si è deciso di gestire le transazioni in modo atomico, considerandole come commit automatici, per evitare una perdita dei dati in caso di errori o eccezioni. I dati, prima dell'effettivo inserimento nel database, vengono gestiti in memoria e, solo al termine di tutte le operazioni, vengono salvati nel database. Questo approccio permette di non impegnare il database con dati parziali, ma, al contempo, una volta validati, la transazione si può considerare come avvenuta, senza dover gestire rollback o commit. All'interno dei repository, è stato aggiunto ai parametri passati ai metodi il "CancellationTok^{en}" [Mica], che permette di interrompere l'esecuzione di una query in caso di timeout o di chiusura dell'applicativo. Questa scelta è stata fatta per evitare di bloccare il database con query che non terminano, e per permettere all'utente di cambiare schermata anche se la query non è terminata, evitando di bloccare l'interfaccia grafica di destinazione.

Capitolo 9

Creazione View

9.1 Utilizzo di Figma per il disegno delle interfacce

La prototipazione delle interfacce grafiche è stata realizzata mediante l'utilizzo del software Figma[Fig]. Figma è una web application che permette di disegnare interfacce grafiche, con la possibilità di creare prototipi interattivi. Un aspetto non secondario è la possibilità di esportare le interfacce sia in formato vettoriale che con codice CSS. Questo aspetto, unito all'utilizzo dell'IA generativa, ha permesso la conversione di interfacce grafiche in codice C# Markup, con un notevole risparmio di tempo. Si è scelto, in un primo momento, di prototipare solo le interfacce principali, quali le schermate di login, ricerca e dettaglio di prodotti, clienti e preventivi. Cercando di mantenere l'obiettivo progettuale orientato al cross-platform, si è deciso di realizzare le interfacce in modo da essere responsive, e con un layout mobile first. Si è quindi cercato di inserire tutti i comandi, escludendo l'utilizzo della tastiera e del mouse, in quanto non sempre disponibili sui dispositivi mobili. Sfruttando la compilazione condizionale, si è cercato di adattare le interfacce grafiche alle diverse piattaforme. Alcuni filtri sono, ad esempio, collassati in un menu a tendina sui dispositivi mobili, mentre su desktop sono sempre visibili. Nelle immagini che seguono si può osservare l'evoluzione dal prototipo in figura 9.1, alla realizzazione finale delle interfacce grafiche in figura 9.2.



Figura 9.1: Prototipo Home Figma

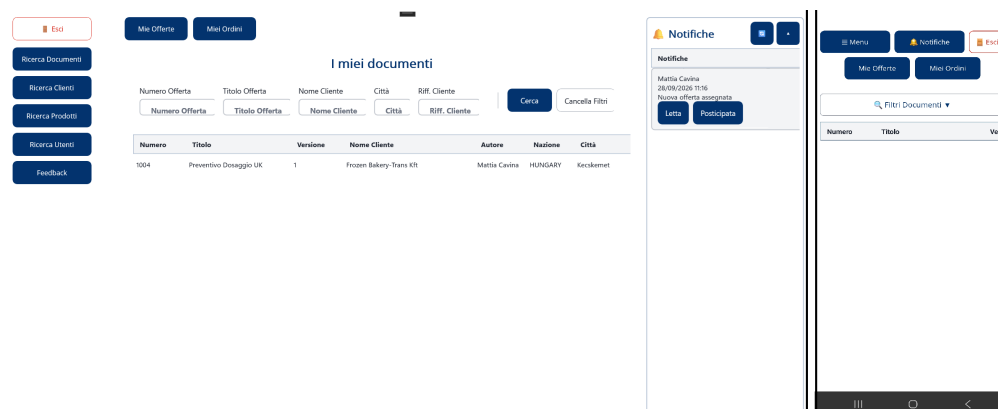


Figura 9.2: Interfaccia Desktop e Mobile

9.2 Creazione delle Factory per le View

A partire dalla versione prototipale, si è deciso fin dall'inizio di realizzare le interfacce grafiche mediante l'utilizzo del pattern Factory. Questo approccio ha

permesso di astrarre la creazione delle interfacce grafiche, unificando i metodi di creazione delle view e permettendo di gestire le interfacce grafiche in un unico package. Le factory principali realizzate sono le seguenti:

- **Colori:** factory che espone i colori utilizzati nell'applicativo, permettendo di centralizzare la gestione dei colori.
- **Componenti:** factory che espone i componenti grafici, come bottoni, etichette e caselle di testo.
- **Dimensioni:** factory che espone le dimensioni dei componenti grafici, come la larghezza e l'altezza tipiche dei componenti.
- **Popup di notifica:** factory che espone i metodi per la creazione di popup di notifica, come messaggi di errore o di conferma.

L'utilizzo centralizzato ha permesso inoltre un ampio riutilizzo dei componenti, oltre alla possibilità di ampliarne le funzionalità senza dover modificare le view che li utilizzano. Ciò è stato possibile gestendo un notevole numero di valori nullable, che permettono di personalizzare i componenti in base alle esigenze.

9.3 Scelta delle classi di ViewModel

Interposti tra le view e i repository, secondo il pattern MVVM consigliato da .NET, si sono creati i ViewModel. Queste classi hanno il compito di esporre i dati alle view, tramite proprietà osservabili, e di gestire la logica di interazione con l'utente. All'interno dei ViewModel, infatti, sono racchiuse proprietà e liste osservabili, che permettono di aggiornare automaticamente le view al variare dei dati. Oltre a ciò, i ViewModel contengono metodi per la gestione degli eventi generati dalle view, richiamando i repository per l'accesso ai dati e gestendo la logica di business. I ViewModel creati non sono stati in rapporto uno a uno con le view, ma piuttosto in base alla loro funzionalità e all'elemento del dominio da gestire. Ad esempio, nella gestione dei prodotti sono stati creati due ViewModel: uno per la gestione semplice dei prodotti e uno specifico per la gestione delle distinte prodotto. Questo approccio, unito all'estendibilità delle classi, ha permesso di ampliare il numero

di view senza dover creare nuovi ViewModel, ma estendendo quelli esistenti. Nel processo di rifattorizzazione si è notata la presenza di comportamenti comuni tra i ViewModel, come la gestione delle liste dedicate ai dropdown, la gestione dei filtri e la gestione dei popup di notifica. Si è quindi creato un ViewModel base, che espone metodi e proprietà comuni a tutti i ViewModel, e questi ultimi, a loro volta, lo estendono. Alcuni dei metodi esposti vengono sottoposti ad override nei ViewModel specifici, per gestire comportamenti particolari.

Capitolo 10

Modulo configurazione prodotto

10.1 Scopo del modulo

Il vero salto di qualità, rispetto all'attuale metodo di creazione delle offerte, è l'introduzione di un modulo di configurazione prodotto. Come citato nell'introduzione [1](#), l'azienda ha un modello di business di tipo ETO. Questo modello comporta la produzione di prodotti realizzati su misura per il cliente, utilizzando però componenti standardizzati. Si è dovuto quindi cercare di realizzare una struttura e un algoritmo che permettessero di configurare un prodotto, senza sapere a priori né da quali regole sia governato né quali prodotti lo compongano. È stato fondamentale anche lo studio di un sistema che guidasse l'utente nella scelta del modello da configurare.

10.2 Algoritmo di configurazione

L'algoritmo implementato si suddivide in tre fasi principali:

- **Fase di scelta del modello:** mediante una scelta a cascata l'utente discende da concetti generali, come lo scopo del componente, fino alla tipologia di prodotto, per arrivare al modello specifico da configurare. Questo modello ad albero guida l'utente.

- **Fase di configurazione:** in questa fase vengono proposti all'utente delle opzioni di configurazione per il modello scelto.
- **Fase di scelta dei componenti:** in base alle scelte precedenti, all'utente vengono proposti i componenti disponibili per la configurazione del prodotto. In questa fase è possibile eliminare quelli non desiderati.
- **Generazione codice:** questa fase finale è la più delicata dell'implementazione dell'algoritmo, in quanto, in base ai codici scelti, si hanno due alternative:
 - **Configurazione esistente:** si ricerca nel database se esiste già una configurazione di quel prodotto con i codici scelti. In caso positivo si propone all'utente di utilizzare quella configurazione.
 - **Configurazione inesistente:** in caso negativo, si genera un nuovo codice di configurazione per il prodotto, che viene salvato nel database.

10.2.1 Scelta macrogruppo da configurare

La realizzazione dell'albero di scelta è stata fatta sfruttando le tabelle già presenti nel database, in particolare le tabelle prodotto, prodotti_composizioni e di lookup. Si sono attribuiti tre nuovi tipi di prodotto, ovvero: iniziale, mappa e macrogruppo. Il prodotto iniziale è la radice dell'albero, e rappresenta il concetto generale del prodotto. Si sono inseriti all'interno delle loro distinte i prodotti di tipo mappa, che rappresentano i concetti intermedi. Infine i prodotti di tipo macrogruppo rappresentano i prodotti finali, sono stati a loro volta inseriti nelle distinte dei prodotti di tipo mappa. Questa struttura permette la realizzazione di un albero ampiamente scalabile, in quanto è possibile aggiungere nuovi prodotti senza dover modificare l'algoritmo. L'albero in questione è osservabile in figura [10.1](#).

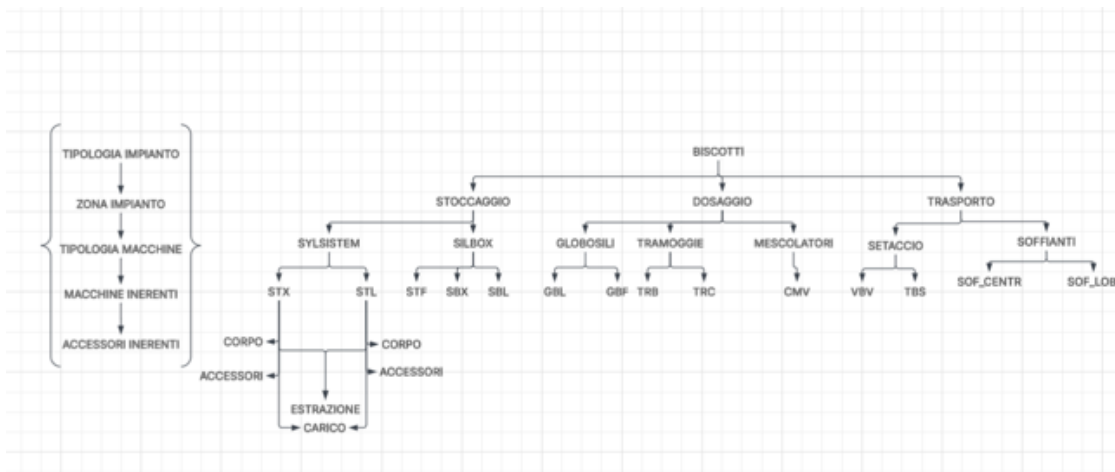


Figura 10.1: Albero di scelta prodotto

10.2.2 Implementazione delle regole

I macrogruppi contengono al loro interno tutti i componenti possibili per la configurazione del prodotto. A ogni componente sono state associate n regole che, se valide, determinano la sua disponibilità nella configurazione. A ogni regola, a sua volta, è associata una proprietà di riferimento, un operatore logico e un valore di riferimento. L'interfaccia utente con le possibili domande di configurazione viene generata filtrando dinamicamente tutte le proprietà di riferimento delle regole, in base al macrogruppo scelto. In questo modo non si ha la necessità di conoscere a priori le regole che governano la configurazione del prodotto. In base alle scelte, come detto in precedenza, si propongono all'utente solo i componenti che rispettano le scelte effettuate. La realizzazione lato database è stata possibile introducendo due nuove tabelle, una per le regole e una di lookup per gli operatori logici. La struttura dinamica del database, implementata nelle prime fasi, ha permesso di realizzare l'implementazione in modo agile. Di seguito, nella figura 10.2, è possibile osservare il modello ER del modulo di configurazione prodotto.

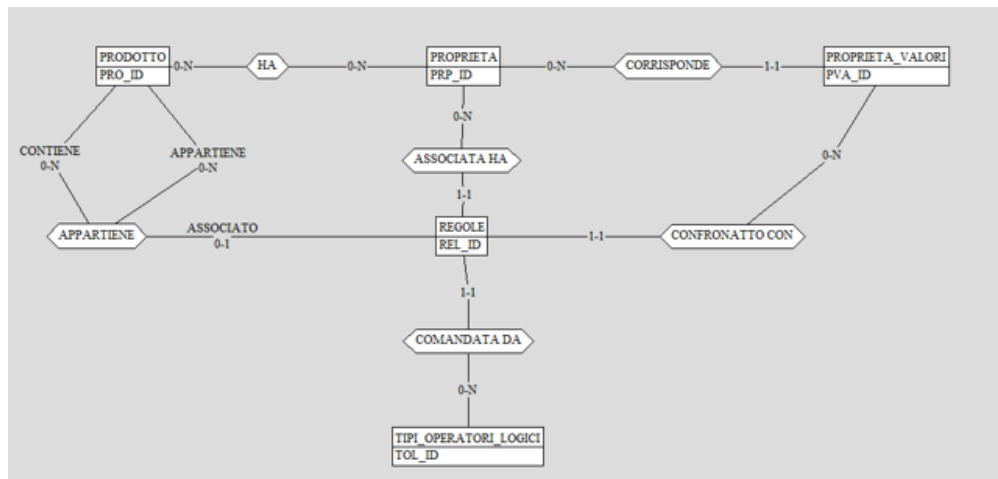


Figura 10.2: Modello ER configuratore

10.3 Interfaccia grafica lato utente ed amministratore

Le interfacce lato utente e amministratore sono profondamente diverse, in quanto, per offrire all'utente un'esperienza di configurazione semplice e guidata, la complessità è stata spostata lato amministratore.

10.3.1 Lato utente

L'esperienza di configurazione lato utente è stata realizzata dopo un focus group con alcuni utenti, in cui si è cercato di capire quale potesse essere il flusso di lavoro più semplice e intuitivo. Si è deciso di proporre le scelte successive solo dopo aver effettuato la scelta precedente, in modo da guidare l'utente. Si è partiti dai concetti generali, come lo scopo del prodotto, fino ad arrivare al modello specifico da configurare. Nella figura ?? è possibile osservare l'interfaccia grafica lato utente.

10.3. INTERFACCIA GRAFICA LATO UTENTE ED AMMINISTRATORE

Configuratore Guidato (CPQ) Reimposta Aggiungi Configurazioni Ingrandisci Chiudi

1. Famiglia / Selezione Iniziale
MICROSOLIDI VUOTASACCHI STOCCAGGIO INTERMEDI FILTRI DOSAGGIO TRASPORTO

2. Sottogruppo / Secondo Livello
ACCESSORI TUBAZIONE SETACCIO SOFFIANTI

3. Dettaglio / Terzo Livello
CDS ACC VST

5. Parametri e Configurazione Macrogruppo

GRANDEZZA
PICCOLA 3LT

TIPO_STELLARE
GRAVITA STD

TUBAZIONE
Ø60

Figura 10.3: Configurazione lato utente

Il risultato finale è visibile nella figura 10.4 sottostante.

Posizione	Codice	Descrizione	Quantità	Unità di Misura	Elimina
	EST00008	ESTRAZ.VST 3LT GRAVITA' DA Ø250	1	-	
	EST00009	ESTRAZIONE VST SOLO PER FONDI FLUIDI	1	-	
	EST00018	COLLEGAMENTO A STELLARE 3LT SOLO GUARN	1	-	
	EST00020	ESTRAZ.VST 3LT SOLO GUARNIZIONE E VITER	1	-	
	EST00026	ESTRAZIONE VALVOLA STELLARE PICCOLA A GR	1	-	
	VST90256	TRONCHETTO DA TUB.Ø60 L=130 INOX COMPL.	1	-	
	VST90257	TRONCHETTO DA TUB.Ø60 L=230 INOX COMPL.	1	-	

Figura 10.4: Risultato configurazione lato utente

Al termine il codice generato o raccolto dal database viene aggiunto ai prodotti dell'offerta, e l'utente può proseguire con la creazione dell'offerta.

10.3.2 Lato amministratore

La creazione delle configurazioni passa dalla composizione dei macrogruppi. Dopo aver compilato i componenti, a ognuno di essi possono essere assegnate una o più regole. L'assegnazione avviene aggiungendo la proprietà di riferimento, l'operatore logico e il valore che deve essere rispettato per rendere disponibile il componente. Di seguito l'interfaccia di aggiunta delle regole ad un componente, visibile in figura ???. Si può notare a destra la scelta delle proprietà mediante filtri e a sinistra la lista delle regole già assegnate al componente.

Proprietà	Operatore	Valore	Agg. Standard	Elimina
GRANDEZZA	=	GRANDE 1ILT	+	
TIPO_STELLARE	=	LINEA STD	+	
TIPO_STELLARE	=	LINEA FRONT	+	
TIPO_STELLARE	=	GRAVITA STD	+	
TIPO_STELLARE	=	GRAVITA FRONT	+	

Nome Proprietà	Tipo Proprietà	Modifica	Elimina
ESTRAZIONE		Modifica	
GRANDEZZA		Modifica	
PORTATA	meccaniche	Modifica	
TIPO_STELLARE		Modifica	
TUBAZIONE		Modifica	

Figura 10.5: Aggiunta regole

Dal focus group con gli utenti che potrebbero gestire la creazione è emerso che testare le regole è un aspetto fondamentale, in quanto permette di capire se le regole inserite sono corrette e se il flusso di configurazione è coerente. Si è quindi creata un'interfaccia di test delle regole, visibile in figura 10.6.

10.3. INTERFACCIA GRAFICA LATO UTENTE ED AMMINISTRATORE

←

Salva Modifiche

Esporta Configurazione

Prova Configurazione

Pulisci Risultato

Codice	Descrizione	QT.	U.M.	N. Regole	Regole
EST00002-11LT	MOTORIZZAZ. VST CEPI 11LT PNA/PA	1	NR	5	Regole
EST00002-3LT-PA	MOTORIZZAZ.VST CEPI 3LT LINEA PA	1	NR	0	Regole
EST00005	ESTRAZ.VST 3LT LINEA DA Ø250 SI PIEDI	1	NR	3	Regole
EST00005-11LT	ESTRAZ.VST 11LT LINEA DA Ø250 SI PIEDI	1	NR	3	Regole
EST00008	ESTRAZ.VST 3LT GRAVITA' DA Ø250	1	NR	3	Regole
EST00008-11LT	ESTRAZ.VST 11LT GRAVITA' DA Ø250	1	NR	3	Regole
EST00009	ESTRAZIONE VST SOLO PER FONDI FLUIDI	1	NR	1	Regole
EST00018	COLLEGAMENTO A STELLARE 3LT SOLO GUARN	1	NR	1	Regole
EST00020	ESTRAZ.VST 3LT SOLO GUARNIZIONE E VITER	1	NR	1	Regole
EST00024	ESTRAZIONE VALVOLA STELLARE PICCOLA DI L	1	NR	3	Regole
EST00024-11LT	ESTRAZIONE VALVOLA STELLARE GRANDE DI L	1	NR	3	Regole
EST00024A	ESTRAZIONE VALVOLA STELLARE PICCOLA DI L	1	NR	3	Regole
EST00024A-11LT	ESTRAZIONE VALVOLA STELLARE GRANDE DI LI	1	NR	3	Regole
EST00026	ESTRAZIONE VALVOLA STELLARE PICCOLA A GR	1	NR	3	Regole
EST00026-11LT	ESTRAZIONE VALVOLA STELLARE GRANDE A GRA	1	NR	3	Regole

GRANDEZZA

PICCOLA 3LT

TIPO_STELLARE

LINEA STD

TUBAZIONE

Ø60

Codice	Descrizione
EST00005	ESTRAZ.VST 3LT LINEA DA Ø250 SI PIEDI
EST00018	COLLEGAMENTO A STELLARE 3LT SOLO GUARN
EST00020	ESTRAZ.VST 3LT SOLO GUARNIZIONE E VITER
EST00024	ESTRAZIONE VALVOLA STELLARE PICCOLA DI L
EST00024A	ESTRAZIONE VALVOLA STELLARE PICCOLA DI L
RAR09000	KIT FLUSSAGGIO COMPLETO SINGOLA UTENZA
VST90256	TRONCHETTO DA TUB.Ø60 L=130 INOX COMPL.
VST90257	TRONCHETTO DA TUB.Ø60 L=230 INOX COMPL.

Figura 10.6: Test regole lato amministratore

Bibliografia

- [Fig] Figma. Figma. <https://www.figma.com/>.
- [MAU] .NET MAUI. .net maui. <https://learn.microsoft.com/it-it/dotnet/maui/what-is-maui?view=net-maui-10.0>.
- [Mica] Microsoft. Cancellationtoken. <https://learn.microsoft.com/it-it/dotnet/api/system.threading.cancellationtoken?view=net-10.0>.
- [Micb] Microsoft. maui.mvvm. <https://learn.microsoft.com/it-it/dotnet/architecture/maui/mvvm>.
- [Micc] Microsoft. Microsoft entity framework. <https://learn.microsoft.com/en-us/ef/>.
- [Midd] Microsoft. Sql server. <https://www.microsoft.com/sql-server>.
- [Mice] Microsoft. Sql server express. <https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb?view=sql-server-ver17>.
- [Micf] Microsoft. Xaml language. <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/>.
- [.NE] .NET. .net. <https://dotnet.microsoft.com/>.
- [NuG] NuGet. Nuget package manager. <https://learn.microsoft.com/it-it/nuget/>.
- [Opea] OpenSource. Communitytoolkit.maui. <https://learn.microsoft.com/it-it/dotnet/communitytoolkit/maui/>.

- [Opeb] OpenSource. Mapster. <https://github.com/MapsterMapper/Mapster>.
- [Rul] Rule Designer. Fusion. <https://www.rulesdesigner.com/it/>.
- [SQL] SQLite. Sqlite. Lightweight database engine, <https://www.sqlite.org/>.
- [Zuc] Zucchetti. Ad-hoc. <https://www.zucchetti.com/it-it/soluzioni/gestionali>.

Acknowledgements

Optional. Max 1 page.